

ML Experiment No 4 Support Vector Machine

Import Libraries

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Import Dataset

```
In [3]: df = pd.read_csv('cell_samples.csv')
```

Describe Dataset

Dataset : https://s3-api.us-gio.objectstorage.softlayer.net/cf-courses-data/CognitiveClass/ML0101ENv3/labs/cell_samples.csv

```
In [4]: df.head()
```

```
Out[4]:
```

	ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
0	1000025	5	1	1	1	2	1	3	1	1	2
1	1002945	5	4	4	5	7	10	3	2	1	2
2	1015425	3	1	1	1	2	2	3	1	1	2
3	1016277	6	8	8	1	3	4	3	7	1	2
4	1017023	4	1	1	3	2	1	3	1	1	2

```
In [5]: df.describe()
```

Out[5]:

	ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BlandChrom	NormNucl	Mit	C
count	6.990000e+02	699.000000	699.000000	699.000000	699.000000	699.000000	699.000000	699.000000	699.000000	699.000
mean	1.071704e+06	4.417740	3.134478	3.207439	2.806867	3.216023	3.437768	2.866953	1.589413	2.689
std	6.170957e+05	2.815741	3.051459	2.971913	2.855379	2.214300	2.438364	3.053634	1.715078	0.951
min	6.163400e+04	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	2.000
25%	8.706885e+05	2.000000	1.000000	1.000000	1.000000	2.000000	2.000000	1.000000	1.000000	2.000
50%	1.171710e+06	4.000000	1.000000	1.000000	1.000000	2.000000	3.000000	1.000000	1.000000	2.000
75%	1.238298e+06	6.000000	5.000000	5.000000	4.000000	4.000000	5.000000	4.000000	1.000000	4.000
max	1.345435e+07	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	4.000

In [5]: `df.columns`

Out[5]: Index(['ID', 'Clump', 'UnifSize', 'UnifShape', 'MargAdh', 'SingEpiSize',
'BareNuc', 'BlandChrom', 'NormNucl', 'Mit', 'Class'],
dtype='object')

In [6]: `df.size`

Out[6]: 7689

In [7]: `df.count()`

```
Out[7]: ID          699
        Clump       699
        UnifSize    699
        UnifShape   699
        MargAdh     699
        SingEpiSize 699
        BareNuc     699
        BlandChrom  699
        NormNucl    699
        Mit         699
        Class       699
        dtype: int64
```

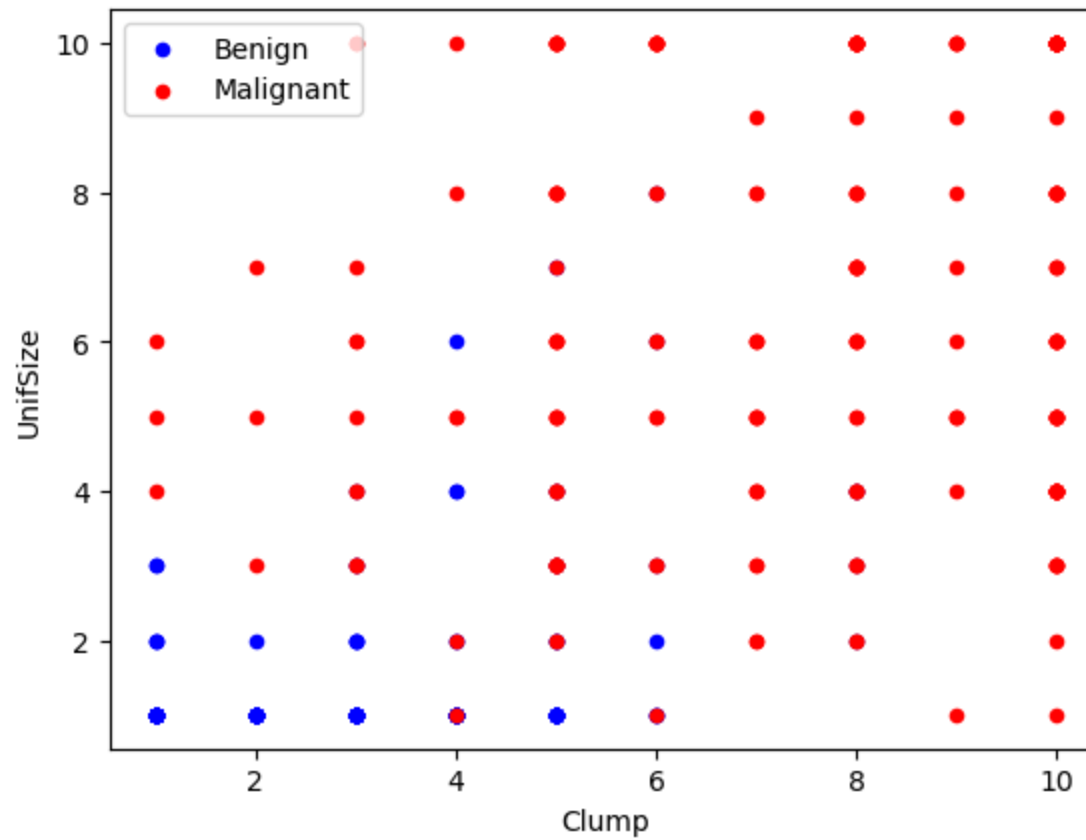
```
In [8]: df['Class'].value_counts()
```

```
Out[8]: Class
2      458
4      241
Name: count, dtype: int64
```

```
In [9]: benign = df[df['Class']==2][0:200]
        malignant = df[df['Class']==4][0:200]

        axes = benign.plot(kind='scatter', x= 'Clump', y = 'UnifSize', color = 'Blue', label = 'Benign')
        malignant.plot(kind = 'scatter', x = 'Clump', y = 'UnifSize', color = 'Red', label = 'Malignant', ax = axes )
```

```
Out[9]: <Axes: xlabel='Clump', ylabel='UnifSize'>
```



Identify Unwanted rows

```
In [10]: df.dtypes
```

```
Out[10]: ID          int64
         Clump       int64
         UnifSize    int64
         UnifShape    int64
         MargAdh     int64
         SingEpiSize int64
         BareNuc     object
         BlandChrom  int64
         NormNucl    int64
         Mit         int64
         Class       int64
         dtype: object
```

```
In [11]: df = df[pd.to_numeric(df['BareNuc'], errors = 'coerce').notnull()]
         df['BareNuc'] = df['BareNuc'].astype('int')
         df.dtypes
```

```
Out[11]: ID          int64
         Clump       int64
         UnifSize    int64
         UnifShape    int64
         MargAdh     int64
         SingEpiSize int64
         BareNuc     int64
         BlandChrom  int64
         NormNucl    int64
         Mit         int64
         Class       int64
         dtype: object
```

Remove Unwanted Columns

```
In [12]: featured_df = df[['Clump', 'UnifSize', 'UnifShape', 'MargAdh', 'SingEpiSize',
                           'BareNuc', 'BlandChrom', 'NormNucl', 'Mit']]

# Independent Variable
x = np.asarray(featured_df)

# Dependent Variable
y = np.asarray(df['Class'])
```

```
x[0:5]
```

```
Out[12]: array([[ 5,  1,  1,  1,  2,  1,  3,  1,  1],
                [ 5,  4,  4,  5,  7, 10,  3,  2,  1],
                [ 3,  1,  1,  1,  2,  2,  3,  1,  1],
                [ 6,  8,  8,  1,  3,  4,  3,  7,  1],
                [ 4,  1,  1,  3,  2,  1,  3,  1,  1]])
```

Divide data for Training and Testing

```
In [13]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state = 4)
X_train.shape
```

```
Out[13]: (546, 9)
```

```
In [14]: X_test.shape
```

```
Out[14]: (137, 9)
```

```
In [15]: y_train.shape
```

```
Out[15]: (546,)
```

```
In [16]: y_test.shape
```

```
Out[16]: (137,)
```

Model SVM

```
In [21]: from sklearn import svm
classifier = svm.SVC(kernel = 'linear', gamma = 'auto', C=2)
classifier.fit(X_train, y_train)
y_predict = classifier.predict(X_test)
```

```
In [25]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_predict))
```

	precision	recall	f1-score	support
2	1.00	0.94	0.97	90
4	0.90	1.00	0.95	47
accuracy			0.96	137
macro avg	0.95	0.97	0.96	137
weighted avg	0.97	0.96	0.96	137